Instalación y trabajo remoto

Requisitos, puesta en marcha de la BeagleBoard y de la PC, y comandos para trabajar a distancia

Banco de programación y depuración remota · octubre de 2026

Repositorio: `https://github.com/ElianUlrich/STM32-bench`

> **Cómo usar esta guía.** Los comandos en recuadro gris se copian y pegan. Fijate siempre **dónde** se escriben: los que empiezan en la PC van en **PowerShell** (el prompt dice `PS C:\>`); los de la placa van en la **terminal de Linux**, a la que se entra por SSH (el prompt dice `ubuntu@arm:~$`). Mezclarlos es el error más común.

Contenido

1. Requisitos de hardware
2. Requisitos de software y cuentas
3. Instalación en la BeagleBoard
4. Instalación en la PC
5. Trabajo remoto desde PowerShell
6. Comandos útiles en la placa
7. Actualizar a una versión nueva
8. Problemas frecuentes

1. Requisitos de hardware

Elemento	Detalle	¿Obligatorio?
BeagleBoard-xM	ARM Cortex-A8, 512 MB de RAM. Otra placa Linux ARM (Raspberry Pi) también sirve con cambios mínimos.	Sí
microSD	4 GB como mínimo; 8 GB o más recomendado (las compilaciones ocupan lugar).	Sí
Fuente de 5 V	Al menos 2,5 A. Con poca corriente se caen los USB.	Sí
Cable de red	Ethernet al router del laboratorio (con acceso a internet).	Sí
ST-Link	V2, V2-1 o V3 (suelto o el de una Nucleo). Para STM32H5 hace falta firmware J24 o posterior (modo dapdirect).	Para STM32
Cables SWD	SWDIO, SWCLK y GND (NRST recomendado). Sin GND común no conecta.	Para STM32
Cable USB de datos	Para DFU, ESP32 y Arduino. Ojo: algunos cables USB-C son sólo de carga.	Según la placa
Adaptador USB-serie 3,3 V	CH340, CP2102 o FT232, para el monitor serie de un STM32 sin puerto serie por USB.	Opcional
Hub USB con fuente	Si se conectan varias placas a la vez.	Recomendado
Teclado y monitor HDMI/DVI	Sólo para el primer arranque si no se puede entrar por la red.	Opcional

2. Requisitos de software y cuentas

Dónde	Qué	Para qué
Placa	Ubuntu 16.04 (imagen rcn-ee) o Debian 11/12 para ARM (armhf)	sistema operativo
Placa	Python 3.9 o superior (en 16.04 se compila con un script)	la aplicación web
Placa	OpenOCD de ST, GDB 12, dfu-util, esptool, avrdude	programar y depurar (los instalan los scripts)
PC	Windows 10/11 con PowerShell y el cliente OpenSSH (viene incluido)	entrar a la placa y copiar archivos
PC	Tailscale para Windows	llegar a la placa desde cualquier red
PC	Un navegador (Edge, Chrome, Firefox)	usar la aplicación
PC	Git para Windows (opcional)	bajar el repositorio y armar el paquete de instalación
PC	STM32CubeIDE (opcional)	compilar firmware; trae arm-none-eabi-gcc y -gdb
Cuentas	Tailscale (gratis para uso personal) y GitHub (el repositorio es privado)	red privada y código

3. Instalación en la BeagleBoard

> Los pasos 3.4 a 3.7 compilan programas en la placa y tardan mucho (de 1 a 6 horas cada uno). Por eso se corren dentro de **tmux**: si se corta la conexión, la compilación sigue. Todos los scripts se pueden volver a correr sin problema.

3.1 Primer acceso por la red local

Con la placa enchufada a la red, buscá su IP en la lista de equipos del router (o conectá monitor y teclado y escribí `ip addr`). Desde PowerShell en una PC de la misma red:

```
ssh ubuntu@192.168.0.27          # usá la IP local de tu placa
```

La primera vez pregunta si confiás en el equipo: escribí `yes`. Después pide la contraseña (la de la imagen; al escribirla no se ve nada, es normal). Cambiala enseguida:

```
passwd
```

3.2 Preparar el sistema (sólo Ubuntu 16.04)

La imagen vieja trae Python 3.5 y su partición no usa toda la microSD. Primero se repara apt y se agranda la partición:

```
sudo dpkg --configure -a
sudo apt update
sudo apt install -y apt-transport-https ca-certificates cloud-guest-utils git tmux
lsblk                                            # ver el nombre del disco y la partición
sudo growpart /dev/mmcblk0 2 && sudo resize2fs /dev/mmcblk0p2
df -h /                                          # comprobar el espacio nuevo
```

3.3 Copiar el código a la placa

Opción A, desde la PC con Git (no hace falta cuenta de GitHub en la placa). En PowerShell, dentro de la carpeta del repositorio:

```
git archive --format=tar.gz -o stm32-bench.tgz HEAD
scp stm32-bench.tgz ubuntu@100.99.235.81:~
```

y en la placa:

```
mkdir -p ~/stm32-bench && tar -xzf ~/stm32-bench.tgz -C ~/stm32-bench && cd ~/stm32-bench
```

Opción B, clonar en la placa (pide usuario y token de GitHub porque el repositorio es privado):

```
git clone https://github.com/ElianUlrich/STM32-bench.git ~/stm32-bench && cd ~/stm32-bench
```

3.4 Python 3.9 y dfu-util (sólo Ubuntu 16.04, ~1-2 h)

```
tmux new -s prep
sudo bash ~/stm32-bench/scripts/prepare_ubuntu1604.sh
```

3.5 OpenOCD de ST (~45-90 min)

```
tmux new -s openocd
sudo bash ~/stm32-bench/scripts/build_openocd.sh
```

3.6 Tailscale

```
curl -fsSL https://tailscale.com/install.sh | sh
sudo tailscale up --hostname=banco-stm32
tailscale ip -4                                  # anotá la IP 100.x.y.z
```

`tailscale up` muestra un enlace: abrilo en la PC e iniciá sesión con tu cuenta. Después, en la consola web de Tailscale (Machines → banco-stm32 → ⋯), elegí **Disable key expiry**: si la clave vence, nadie puede entrar a la placa sin ir hasta ella.

3.7 La aplicación

```
sudo bash ~/stm32-bench/scripts/install.sh
```

Pregunta el **usuario y la contraseña de la web** (mínimo 10 caracteres; usá una distinta de la de Linux). Instala las dependencias, esptool y avrdude, crea el usuario del servicio, las reglas USB y el servicio que arranca solo. Al final dice `Servicio en marcha.` y la dirección para abrir.

3.8 GDB para la pestaña Depurar (opcional, 3-6 h)

```
tmux new -s gdb
sudo bash ~/stm32-bench/scripts/build_gdb.sh
arm-none-eabi-gdb --version                       # al terminar: GNU gdb (GDB) 12.1
```

3.9 Verificar

```
systemctl status stm32-bench --no-pager           # tiene que decir active (running)
tailscale status                                  # la placa y tu PC conectadas
```

Desde la PC, con Tailscale activo, abrí `http://100.99.235.81:8080` (o `http://banco-stm32:8080`) e ingresá con el usuario de la web.

4. Instalación en la PC

4.1 Tailscale

Descargalo de `https://tailscale.com/download/windows`, instalalo e iniciá sesión con la **misma cuenta** que usaste en la placa. En el ícono de la bandeja tiene que figurar banco-stm32 como conectado. Comprobación en PowerShell:

```
tailscale status
ping 100.99.235.81
```

4.2 Cliente SSH

Windows 10 y 11 lo traen. Comprobalo:

```
ssh -V
```

Si dice que no existe, instalalo desde Configuración → Aplicaciones → Características opcionales → Cliente OpenSSH, o en una PowerShell abierta como administrador:

```
Add-WindowsCapability -Online -Name OpenSSH.Client~~~~0.0.1.0
```

4.3 Git y el repositorio (opcional)

Instalá Git para Windows (`https://git-scm.com`) y bajá el repositorio:

```
cd $env:USERPROFILE\Desktop
git clone https://github.com/ElianUlrich/STM32-bench.git Debugger
```

4.4 STM32CubeIDE (opcional)

Sirve para compilar firmware y trae el GDB para ARM. El GDB queda en una carpeta como `C:\ST\STM32CubeIDE_1.19.0\STM32CubeIDE\plugins\…gnu-tools-for-stm32…\tools\bin`. El script `firmware\h503_leds\depurar.bat` ya lo busca ahí.

4.5 Probar la aplicación en la PC, sin placa (opcional)

```
powershell -ExecutionPolicy Bypass -File scripts\dev_run.ps1
```

Abre la app en modo simulación en `http://127.0.0.1:8080`; la contraseña aparece en la consola.

5. Trabajo remoto desde PowerShell

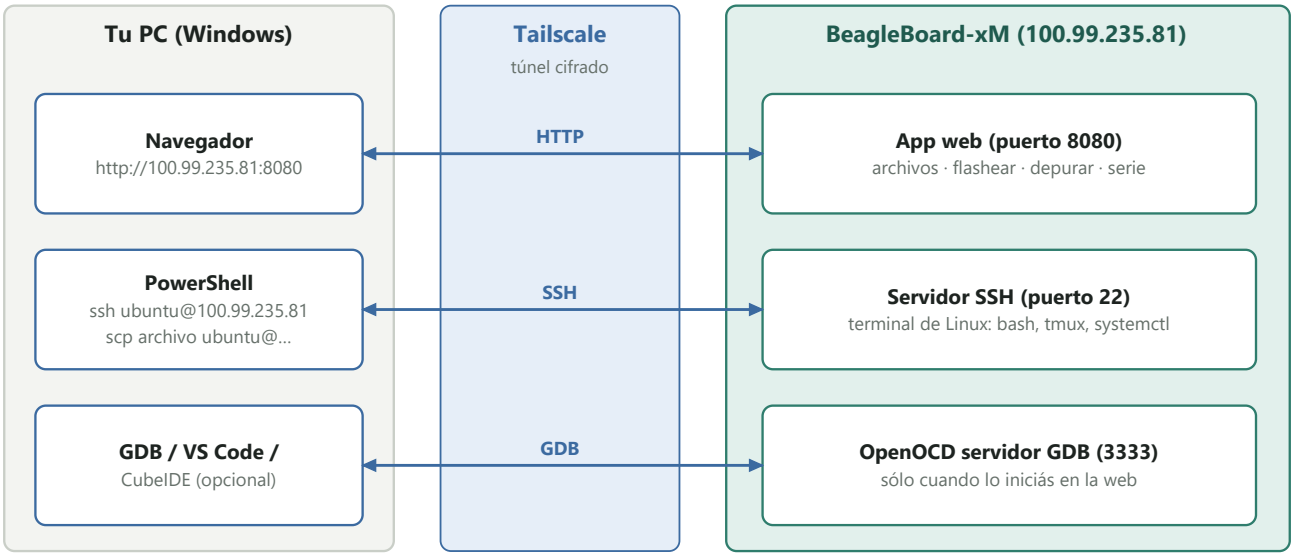


Figura 1 — Las tres formas de trabajar con el banco desde la PC.

5.1 Entrar a la terminal de Linux de la placa

```
ssh ubuntu@100.99.235.81              # o:  ssh ubuntu@banco-stm32
```

Cuando el prompt cambia a `ubuntu@arm:~$` ya estás en la placa: todo lo que escribas corre en Linux. Para volver a PowerShell: `exit` (o Ctrl+D).

> **PowerShell no es Linux.** En PowerShell 5 no existe `&&` para encadenar comandos (usá `;`), ni `df`, `grep` o `sudo`. Si un comando de esta guía da error de "no se reconoce", seguramente lo escribiste en la PC en lugar de en la placa.

5.2 Entrar sin escribir la contraseña cada vez (clave SSH)

Se hace una sola vez. En PowerShell, creá la clave (Enter a todas las preguntas) y copiala a la placa:

```
ssh-keygen -t ed25519
scp $env:USERPROFILE\.ssh\id_ed25519.pub ubuntu@100.99.235.81:~/clave.pub
```

Después, en la placa (es la última vez que pide la contraseña):

```
mkdir -p ~/.ssh && cat ~/clave.pub >> ~/.ssh/authorized_keys
chmod 700 ~/.ssh && chmod 600 ~/.ssh/authorized_keys && rm ~/clave.pub
```

5.3 Un nombre corto para la placa

Creá el archivo `C:\Users\<tu usuario>\.ssh\config` (sin extensión) con:

```
Host banco
    HostName 100.99.235.81
    User ubuntu
```

Desde ese momento alcanza con `ssh banco`, `scp archivo banco:~`, etc.

5.4 Copiar archivos entre la PC y la placa

```
# de la PC a la placa (queda en la carpeta personal, ~)
scp C:\ruta\archivo.elf ubuntu@100.99.235.81:~
# de la placa a la carpeta actual de la PC
scp ubuntu@100.99.235.81:/var/lib/stm32-bench/history.jsonl .
# una carpeta entera
scp -r C:\ruta\carpeta ubuntu@100.99.235.81:~
```

Para el firmware conviene subirlo desde la web (pestaña Archivos), que además lo asigna a su placa.

5.5 Ejecutar un comando en la placa sin quedarse adentro

```
ssh ubuntu@100.99.235.81 "systemctl status stm32-bench --no-pager"
ssh ubuntu@100.99.235.81 "uptime; df -h /; free -m"
ssh -t ubuntu@100.99.235.81 "sudo systemctl restart stm32-bench"
```

Con `-t` se puede usar sudo, porque deja que la placa pida la contraseña.

5.6 Trabajos largos que sobreviven a un corte: tmux

Comando (en la placa)	Qué hace
`tmux new -s nombre`	abre una sesión nueva con ese nombre
Ctrl+B y después D	sale de la sesión sin cortarla (sigue corriendo)
`tmux ls`	lista las sesiones abiertas
`tmux attach -t nombre`	vuelve a entrar a una sesión
`exit`	dentro de la sesión: la cierra

5.7 Depurar con GDB desde la PC

Con el servidor GDB iniciado en la web (botón Iniciar, tiene que decir “Escuchando”):

```
C:\Users\<tu usuario>\Desktop\Debugger\firmware\h503_leds\depurar.bat
# o a mano:
arm-none-eabi-gdb programa.elf -ex "set remotetimeout 10" `
    -ex "target extended-remote 100.99.235.81:3333"
```

No abras `100.99.235.81:3333` en el navegador: no es una página web y ocupa la única conexión GDB.

5.8 Túnel SSH (sólo si no podés usar Tailscale)

Si estás en la misma red local, un túnel lleva la web a tu PC sin abrir puertos:

```
ssh -L 8080:localhost:8080 ubuntu@192.168.0.27
# mientras esa ventana esté abierta:  http://localhost:8080
```

6. Comandos útiles en la placa

Comando	Para qué
`systemctl status stm32-bench`	estado del servicio de la web
`sudo systemctl restart stm32-bench`	reiniciarlo (por ejemplo después de editar la configuración)
`journalctl -u stm32-bench -f`	ver el log del servicio en vivo (Ctrl+C para salir)
`sudo nano /etc/stm32-bench/chips.yaml`	editar las placas (Ctrl+O guarda, Ctrl+X sale)
`lsusb`	ver qué hay conectado por USB (ST-Link 0483:3748, DFU 0483:df11…)
`ls /dev/ttyUSB* /dev/ttyACM*`	puertos serie disponibles
`dmesg | tail -25`	últimos mensajes del sistema (USB que entra y sale, red)
`tailscale status`	equipos de la red privada
`sudo systemctl restart tailscaled`	reconectar Tailscale
`ip link show eth0`	estado del cable de red (NO-CARRIER = desconectado)
`uptime; df -h /; free -m`	tiempo encendida, disco y memoria
`sudo poweroff`	apagar (esperá unos 30 s antes de desenchufar)
`sudo reboot`	reiniciar

7. Actualizar a una versión nueva

Cuando hay cambios en el repositorio. En PowerShell, en la carpeta del repositorio:

```
git pull
git archive --format=tar.gz -o stm32-bench.tgz HEAD
scp stm32-bench.tgz ubuntu@100.99.235.81:~
```

En la placa:

```
rm -rf ~/stm32-bench && mkdir ~/stm32-bench
tar -xzf ~/stm32-bench.tgz -C ~/stm32-bench
cd ~/stm32-bench && sudo bash scripts/install.sh
```

install.sh conserva el usuario, la contraseña y la configuración. Si la configuración del repositorio cambió, deja la nueva al lado como `chips.yaml.dist` / `bench.yaml.dist`. Para usarla:

```
sudo cp /etc/stm32-bench/chips.yaml.dist /etc/stm32-bench/chips.yaml
sudo systemctl restart stm32-bench
```

Después recargá la web con Ctrl+F5.

8. Problemas frecuentes

Síntoma	Causa probable y solución
`ssh: connect ... timed out`	Tailscale apagado en la PC o en la placa, o cable de red flojo. Revisá `tailscale status` en la PC; si la placa no aparece, entrá por la red local y corré `sudo systemctl restart tailscaled`.
`REMOTE HOST IDENTIFICATION HAS CHANGED`	Se reinstaló el sistema de la placa. En PowerShell: `ssh-keygen -R 100.99.235.81` y volvé a entrar.
`client_loop: send disconnect`	Se cortó la conexión (la placa muy ocupada compilando, o la red). Volvé a entrar; lo que estaba en tmux sigue corriendo.
"df / grep / sudo no se reconoce"	Lo escribiste en PowerShell: primero `ssh ubuntu@...`.
`&& no es un separador válido`	PowerShell 5 no tiene &&: usá ; o un comando por línea.
`Sorry, try again`	Contraseña de sudo mal escrita (es la del usuario ubuntu).
La web no carga	Tailscale activo en la PC; `systemctl status stm32-bench` en la placa; mirá `journalctl -u stm32-bench -n 50`.
El ST-Link no conecta al chip	Falta el GND común, o el chip está sin alimentación. El H5 necesita la interfaz stlink-dap.
La pestaña Depurar no muestra el código	Subiste sólo el .elf: subí el proyecto en .zip con las fuentes.
La placa no vuelve después de apagarla	No se puede encender a distancia: hay que desenchufar y volver a enchufar la alimentación.